

StockSense: Recap — Phases 0 through 5b

Part 1: Phase-by-Phase Summary

Phase 0 — Environment & Repo Setup

What we did: Installed Python, VSCode, Git, Docker Desktop. Created a git repo, `.gitignore`, a virtual environment, and a starter `requirements.txt`.

Files created: `.gitignore`, `.venv/` (not committed), `requirements.txt`

Learnings: Virtual environments isolate dependencies per-project. `.gitignore` keeps secrets/junk/generated files out of version control. Git hygiene from commit #1, not bolted on later.

Phase 1 — Project Skeleton & Config

What we did: Built the folder structure (`ingestion/`, `db/`, `dags/`, `api/`, `ui/`, `ml/`, `tests/`). Set up `.env` + `.env.example` for secrets, and `config.py` to load them.

Files created: `config.py`, `.env`, `.env.example`, empty package folders with `__init__.py`, `README.md`

Learnings: Secrets never get hardcoded or committed — `.env` holds real values (gitignored), `.env.example` documents *which* variables exist (committed, no real values). `__init__.py` marks a folder as an importable Python package.

Phase 2 — Data Ingestion Scripts

What we did: Wrote standalone scripts to pull S&P 500 price data (`yfinance`) and market news (NewsAPI), saving to local CSV/JSON initially.

Files created: `ingestion/fetch_prices.py`, `ingestion/fetch_news.py`

Learnings: Per-item `try/except` so one bad ticker/article doesn't kill a whole batch run. `logging` instead of `print` (levels, timestamps — and it's what Airflow will display later). `timeout=` on every network call. `response.raise_for_status()` turns silent bad responses into catchable exceptions.

Phase 3 — Database Layer

What we did: Ran Postgres in Docker. Defined SQLAlchemy models (`Price`, `NewsArticle`). Built upsert logic so ingestion is safe to rerun. Wired the Phase 2 scripts to write into Postgres instead of files.

Files created: `docker-compose.yml`, `db/database.py`, `db/models.py`, `db/init_db.py`, `db/repository.py`

Learnings: Docker volumes persist data beyond a container's lifecycle.

`UniqueConstraint`/`unique=True` + `ON CONFLICT` upserts make ingestion **idempotent** — safe to rerun or retry without duplicating data. `try/except/rollback/finally` is the standard DB transaction-safety pattern. `create_all()` creates new tables but never alters existing ones (a real limitation we hit again in 5b).

Phase 4 — Airflow

What we did: Dockerized Airflow (custom image + Compose services for webserver/scheduler/its own metadata DB). Wrote a DAG scheduling the Phase 2/3 ingestion logic daily, plus a data quality check task.

Files created: `Dockerfile.airflow`, `dags/daily_ingestion_dag.py`; heavily extended `docker-compose.yml`

Learnings (including real bugs we hit and fixed):

- Containers reach each other by **service name** (`postgres`), not `localhost` — `localhost` inside a container refers to that container itself.
 - `load_dotenv()` doesn't override env vars already set by the environment (e.g., by Compose) — it only fills in what's missing. This is why Compose-level `environment:` values silently "win" over `.env` file values inside containers.
 - `PYTHONPATH` controls where Python looks for importable packages — it's not automatically inherited from wherever your code happens to live, especially across container boundaries.
 - Webserver and scheduler need a matching `AIRFLOW__WEBSERVER__SECRET_KEY` to share logs correctly.
 - Task dependency graphs (`[task_a, task_b] >> task_c`) declare *what depends on what*, not just a sequential list of steps.
 - `retries` + `retry_delay` make transient failures (a flaky API) self-healing, relying on the idempotency we built in Phase 3.
 - Hard-fail vs. soft-warn is a judgment call — price data missing is unambiguous ("fail loudly"); low/zero news volume can be legitimate ("log a warning, don't fail the run").
-

Phase 5 — Baseline ML Model

What we did: Backfilled 2 years of price history (separate from daily incremental ingestion). Built price-based features (returns, moving averages, volatility) with next-day up/down labels. Trained a logistic regression using a **time-based** train/test split, evaluated against a naive "always predict up" baseline.

Files created: `ml/backfill_history.py`, `ml/features.py`, `ml/train.py`

Learnings:

- Training needs a fundamentally different data volume/shape than daily serving — a one-time backfill script is a normal, separate concept from incremental ingestion.
 - Rolling features must be computed **per-ticker** (grouped), not across the whole concatenated dataset — otherwise you blend unrelated tickers' histories together.
 - Labels built with `.shift(-1)` must be computed *before* dropping NaN rows, and every feature must only use data available *as of* the prediction date — this is how you avoid **lookahead bias**, the most common and dangerous mistake in time-series ML.
 - Splitting by **date**, not randomly, is mandatory for time-series — a random split can leak future information into training.
 - Reporting a naive baseline alongside your model's metrics is what makes an evaluation honest, especially on a problem (market direction) that's inherently close to a coin flip.
-

Phase 5b — LLM-Scored News Sentiment

What we did: Distinguished "LLM feature extraction" from "RAG" (different techniques). Used Gemini 2.5 Flash-Lite with structured output (Pydantic schema) to score news articles for market sentiment, as a new daily Airflow task. Added the resulting score as a new model feature, aggregated as market-wide daily sentiment.

Files created/edited: `ml/sentiment.py`; edited `db/models.py` (new columns), `dags/daily_ingestion_dag.py` (new task), `ml/features.py` (merge sentiment in), `ml/train.py` (new feature column), `docker-compose.yml` (new env var)

Learnings:

- RAG (retrieval to answer a question) and LLM feature extraction (scoring/labeling data to feed a model) solve different problems — worth being precise about which one a task actually calls for.
 - `response_schema` with a Pydantic model gets reliable structured JSON out of an LLM, instead of parsing free-form text.
 - Adding a genuinely new column to an existing table needs a manual `ALTER TABLE` (or Alembic) — `create_all()` won't retroactively modify tables that already exist.
 - Storing every field a system produces (`sentiment_score` and `market_relevant`), and deciding how to filter/use it downstream, is more flexible than discarding data early based on today's assumptions about what you'll need.
 - Missing feature values (days with no scored news) need a deliberate default (`fillna(0.0)` = neutral) rather than dropping rows — especially when a new feature's historical coverage is much shorter than the rest of your dataset.
-

Part 2: Why This Order?

This is the part that builds real intuition, so let's walk through the actual dependency logic — not just "phase 1 before phase 2 because I said so."

You can't test ingestion logic against a real database until a database exists. Phase 2 (ingestion scripts) intentionally wrote to CSV/JSON files first — not because that's the end goal, but so you could validate "does this script correctly fetch and parse data" as a fully separate question from "does this script correctly talk to a database." Debugging both at once, from scratch, is much harder than debugging them one at a time.

You can't orchestrate a pipeline that doesn't exist yet. Airflow's whole job is scheduling and monitoring *existing* logic — there was nothing for it to orchestrate until Phases 2–3 produced working, callable functions (`fetch_prices()`, `upsert_prices()`, etc.). This is why the DAG file's tasks are thin wrappers that import and call functions we already wrote and tested — Airflow orchestrates, it doesn't contain business logic itself.

You can't train a model on data you haven't collected yet. Phase 5 depended on Phase 3 (a database to query) and indirectly on Phase 4 (a running pipeline that will keep feeding it fresh data going forward) — but training itself needed *more history* than daily ingestion provides, which is why backfilling was its own explicit step, not an afterthought.

The baseline came before the LLM enhancement, on purpose. Phase 5 (price-only) gives you a real number to compare against. Phase 5b (adding sentiment) is only meaningful *because* you can say "accuracy went from X to Y" — building the LLM feature first, with nothing to compare it to, would've made it impossible to tell whether it actually helped.

General pattern underneath all of this: each phase built something the *next* phase could treat as a stable, already-working dependency — a callable function, a populated table, a running container — rather than everything being built and debugged simultaneously. This is really the core intuition of software architecture: **build and validate one layer at a time, so failures are easy to localize** to the layer you just touched, not "somewhere in everything I wrote today."

Part 3: Why Split Into These Specific Files?

The guiding principle across every phase: **files are split along the lines of "what would need to change independently of what else."** A few concrete examples from your own code:

`db/database.py` vs. `db/models.py` vs. `db/repository.py`

- `database.py` = *connection setup* (engine, session factory). This never changes unless your DB technology or credentials change.
- `models.py` = *schema* (what tables/columns exist). This changes whenever your data shape changes (like adding `sentiment_score`).

- `repository.py` = *operations* (upsert logic). This changes whenever your write patterns change.

These three concerns genuinely change for different reasons, at different times — bundling them into one file would mean every schema tweak risks touching connection logic you didn't mean to touch.

`ingestion/` vs. `ml/` vs. `api/`

- `ingestion/` = getting external data *in*.
- `ml/` = turning stored data into features and predictions.
- `api/` (coming in Phase 6) = exposing data/predictions *out*, to a UI or any other consumer.

Notice the direction: data flows `ingestion` → `database` → `ml` → `api` → `ui`. Each folder is a distinct *stage* of that pipeline. This is why, back in Phase 5b, we were careful that the UI (eventually) never touches the database directly — it goes through the API, which goes through the same `db/` layer everything else uses. One shared data-access layer, multiple consumers.

Why `dags/daily_ingestion_dag.py` is so thin Look at how little logic actually lives in the DAG file itself — it mostly just imports functions from `ingestion/`, `db/`, and `ml/` and wires them into a schedule. This is deliberate: **the DAG describes *when and in what order* things run; it doesn't contain the actual logic of *what* those things do.** If you ever wanted to run `fetch_prices()` outside of Airflow (which you did, manually, in Phase 2 before Airflow even existed), you could — because the logic doesn't live inside Airflow's world, Airflow just calls into it.

The test this pattern passes: for any file in your project, you should be able to answer "if I needed to change X, which single file would I touch, and would that change accidentally affect something unrelated?" If the answer is "I'd have to touch this one big file that does five different things," that's usually a sign it should be split further. If the answer is "I'd touch exactly this one small file, and nothing else needs to know," that's the structure working as intended — which is genuinely the whole point of splitting code into files/modules in the first place, not just following a template because it looked standard.